



HOSTEL ROOM BOOKING AND FEES MANAGEMENT SYSTEM

GROUP COURSEWORK REPORT

Group Members

Member	Registration No.	Contribution
Namakula Waludah	VU-DIT-2603-0250-EVE	Hostel blocks, rooms and room allocation
Obentu Elivis	VU-BCS-2603-203-EVE	Student registration and student search
Nyango George	VU-BIT-2603-3365	Fee payment and balance calculation
Mukiza Eugene	VU-DIT-2603-2687-EVE	Reports, file persistence and system integration

1. Introduction

The Hostel Room Booking and Fees Management System is a Python-based system designed to help a university manage student hostel rooms and fee payments.

The system replaces the paper-based method with a simple computerized system that can register students, allocate rooms, record payments, search records and produce reports.

2. System Objectives

The main objectives of the system are to:

- Register hostel students.
- Allocate students to available rooms.
- Prevent rooms from exceeding their capacity.
- Record full and partial hostel fee payments.
- Calculate outstanding balances.
- Search students by name or registration number.
- Generate hostel occupancy reports.
- Identify students with unpaid balances.
- Save records so that they remain available after the program closes.

3. Python Source Code

The following is the complete Python program for the system.

```
import json
import os
```

```
FILE = "hostel_data.txt"
```

```

hostels = {
    "Block A": {"A101": [4, []], "A102": [4, []], "A103": [2, []]},
    "Block B": {"B101": [4, []], "B102": [4, []], "B103": [2, []]},
    "Block C": {"C101": [3, []], "C102": [3, []], "C103": [2, []]}
}

students = {}

def save():
    try:
        with open(FILE, "w") as f:
            json.dump({"hostels": hostels, "students": students}, f)
        print("Records saved.")
    except OSError:
        print("Could not save records.")

def load():
    global hostels, students
    if not os.path.exists(FILE):
        return

    try:
        with open(FILE, "r") as f:
            data = json.load(f)
            hostels = data["hostels"]
            students = data["students"]
    except (OSError, json.JSONDecodeError, KeyError):
        print("Saved file is damaged.")
        students = {}

def overview():
    print("\n===== OCCUPANCY OVERVIEW =====")

```

```

for block, rooms in hostels.items():
    used = sum(len(r[1]) for r in rooms.values())
    total = sum(r[0] for r in rooms.values())
    print(f"{block}: {used}/{total} occupied")

```

```

def register():
    print("\n===== STUDENT REGISTRATION =====")
    reg = input("Registration number: ").strip()
    name = input("Student name: ").strip()

```

```

    if not reg or not name or reg in students:
        print("Invalid or duplicate student details.")
        return

```

```

    print("Blocks:", ", ".join(hostels))
    block = input("Block: ").strip()

```

```

    if block not in hostels:
        print("Invalid block.")
        return

```

```

    print("Rooms:", ", ".join(hostels[block]))
    room = input("Room: ").strip()

```

```

    if room not in hostels[block]:
        print("Invalid room.")
        return

```

```

    r = hostels[block][room]

```

```

    if len(r[1]) >= r[0]:
        print("Room is full.")

```

```

        return

    try:
        fee = float(input("Total fee: "))

        if fee < 0:
            raise ValueError

    except ValueError:
        print("Invalid fee.")
        return

    students[reg] = {
        "name": name,
        "block": block,
        "room": room,
        "fee": fee,
        "payments": []
    }

    r[1].append(reg)

    print("Student registered successfully.")


def payment():
    print("\n===== FEE PAYMENT =====")

    reg = input("Registration number: ").strip()

    if reg not in students:
        print("Student not found.")

```

```

    return

s = students[reg]
paid = sum(s["payments"])
balance = s["fee"] - paid

print(f"Student: {s['name']}")
print(f"Outstanding balance: {balance:.2f}")

if balance <= 0:
    print("Fees are fully paid.")
    return

try:
    amount = float(input("Payment amount: "))

    if amount <= 0 or amount > balance:
        print("Invalid payment amount.")
        return

except ValueError:
    print("Enter a valid amount.")
    return

s["payments"].append(amount)

new_balance = s["fee"] - sum(s["payments"])

print(f"New balance: {new_balance:.2f}")

def search():
    key = input("\nEnter name or registration number: ").lower()

```

```
found = False
```

```
for reg, s in students.items():
```

```
    if key == reg.lower() or key in s["name"].lower():
```

```
        paid = sum(s["payments"])
```

```
        balance = s["fee"] - paid
```

```
        print(f"\nName: {s['name']}")
```

```
        print(f"Registration: {reg}")
```

```
        print(f"Room: {s['block']} {s['room']}")
```

```
        print(f"Fee: {s['fee']:.2f}")
```

```
        print(f"Paid: {paid:.2f}")
```

```
        print(f"Balance: {balance:.2f}")
```

```
    found = True
```

```
if not found:
```

```
    print("Student not found.")
```

```
def report():
```

```
    print("\n===== OCCUPANCY REPORT =====")
```

```
    for block, rooms in hostels.items():
```

```
        print(f"\n{block}")
```

```
        for room, r in rooms.items():
```

```
            print(f"{room}: {len(r[1])}/{r[0]} occupied")
```

```

def defaulters():
    try:
        limit = float(input("\nEnter balance threshold: "))

    except ValueError:
        print("Invalid amount.")
        return

    print("\n===== FEE DEFAULTERS =====")

    found = False

    for reg, s in students.items():

        balance = s["fee"] - sum(s["payments"])

        if balance > limit:
            print(
                f"{s['name']} | {reg} | "
                f"Balance: {balance:.2f}"
            )
            found = True

    if not found:
        print("No defaulters found.")

def main():

    load()
    overview()

```

```

while True:

    print("""
===== HOSTEL MANAGEMENT SYSTEM =====
1. Register student
2. Record payment
3. Search student
4. Occupancy report
5. Fee defaulters
6. Save and exit
""")

    choice = input("Choose: ").strip()

    if choice == "1":
        register()

    elif choice == "2":
        payment()

    elif choice == "3":
        search()

    elif choice == "4":
        report()

    elif choice == "5":
        defaulters()

    elif choice == "6":
        save()

```

```

        print("System closed.")
        break

    else:
        print("Invalid choice.")
if __name__ == "__main__":
    main()

```

4. Data Setup

The system contains **three hostel blocks**, each with rooms having fixed capacities.

Hostel Block	Room	Capacity
Block A	A101	4
Block A	A102	4
Block A	A103	2
Block B	B101	4
Block B	B102	4
Block B	B103	2
Block C	C101	3
Block C	C102	3
Block C	C103	2

Note: The system displays the occupancy of each block when it starts.

5. Sample Student Data

The following sample records were used to demonstrate the system.

Registration No.	Student Name	Block	Room	Total Fee
ST001	Peter Ouma	Block A	A101	600,000
ST002	Grace Achieng	Block A	A101	600,000
ST003	Brian Kato	Block B	B101	600,000
ST004	Jane Nakato	Block C	C101	600,000

6. Sample Fee Payments

The system supports more than one payment for the same student.

Student	Payment	Total Paid	Balance
Peter Ouma	350,000	350,000	250,000
Grace Achieng	450,000	450,000	150,000
Brian Kato	500,000	500,000	100,000
Jane Nakato	0	0	600,000

7. Room Allocation

When a student is registered, the system checks the selected room capacity before allocation.

For example, A101 has a capacity of four students. If four students are already allocated to A101, another student cannot be added and the system displays:

When a student is successfully allocated, the student's registration number is added to the room record and the occupancy increases.

8. Student Search

The search function allows the warden to find a student using either:

- Student name
- Registration number

A successful search displays the student's name, registration number, hostel block, room, total fee, amount paid and outstanding balance.

9. Occupancy Report

The occupancy report shows the number of students currently occupying each room.

Example:

===== OCCUPANCY REPORT =====

Block A

A101: 2/4 occupied

A102: 0/4 occupied

A103: 0/2 occupied

Block B

B101: 1/4 occupied

B102: 0/4 occupied

B103: 0/2 occupied

Block C

C101: 1/3 occupied

C102: 0/3 occupied

C103: 0/2 occupied

This helps the warden identify rooms that still have available spaces.

10. Fee Defaulters

The system can identify students whose outstanding balance is greater than a selected amount.

For example, if the threshold is **200,000**, the system identifies:

Peter Ouma | ST001 | Balance: 250000.00

Jane Nakato | ST004 | Balance: 600000.00

This makes it easier for the hostel administration to follow up unpaid fees.

11. File Persistence

The system saves student, room and payment records in:

hostel_data.txt

The records are automatically loaded when the program starts.

If the file is missing, the system starts normally. If the file is damaged or contains invalid information, the program displays an error message instead of crashing.

12. Menu-Driven System

The main menu brings all system functions together.

===== HOSTEL MANAGEMENT SYSTEM =====

1. Register student
2. Record payment
3. Search student
4. Occupancy report
5. Fee defaulters
6. Save and exit

The menu continues running until the user selects **Save and exit**.

13. System Design

The system uses Python dictionaries and lists to store hostel and student information.

Functions were used to separate the major operations such as registration, payments, searching, reporting, saving and loading data.

The system also uses JSON file storage so that information can remain available between different program sessions.

14. Group Division of Work

Group Member	Registration Number	Main Responsibility
Namakula Waludah	VU-DIT-2603-0250-EVE	Hostel blocks, rooms and room allocation
Obentu Elivis	VU-BCS-2603-203-EVE	Student registration and student search
Nyango George	VU-BIT-2603-3365	Fee payment and balance calculation
Mukiza Eugene	VU-DIT-2603-2687-EVE	Reports, file persistence and system integration

Each member contributes to the development and testing of the complete system.

15. Challenges and Solutions

Challenge 1: Room capacity

A room could accidentally receive more students than its capacity.

Solution: The program checks the current occupancy before adding a student.

Challenge 2: Partial payments

Students may pay hostel fees in several installments.

Solution: Payments are stored in a list and added together to calculate the remaining balance.

Challenge 3: Loss of records

Records could disappear when the program closes.

Solution: Student and hostel information is saved to a file and loaded when the system starts.

Challenge 4: Damaged files

A damaged data file could cause the program to stop.

Solution: Error handling was included so the system can continue without crashing.

16. Screenshots for the Answer Sheet

The project demonstration can be represented using screenshots showing:

Screenshot 1: Program startup and occupancy overview.

```
C:\Program Files\WindowsAp  X  +  v

===== OCCUPANCY OVERVIEW =====
Block A: 0/10 occupied
Block B: 0/10 occupied
Block C: 0/8 occupied

===== HOSTEL MANAGEMENT SYSTEM =====
1. Register student
2. Record payment
3. Search student
4. Occupancy report
5. Fee defaulters
6. Save and exit

Choose: |
```

Screenshot 2: Student registration and successful room allocation.

```
Choose: 1

===== STUDENT REGISTRATION =====
Registration number: VU001
Student name: John Okello
Blocks: Block A, Block B, Block C
Block: Block A
Rooms: A101, A102, A103
Room: A101
Total fee: 500000
Student registered successfully.
```

Screenshot 3: Fee payment and updated outstanding balance.

```
Choose: 2

===== FEE PAYMENT =====
Registration number: VU001
Student: John Okello
Outstanding balance: 500000.00
Payment amount: 200000
New balance: 300000.00
```

Screenshot 4: Student search result.

```
Choose: 3

Enter name or registration number: VU001

Name: John Okello
Registration: VU001
Room: Block A A101
Fee: 500000.00
Paid: 200000.00
Balance: 300000.00
```

Screenshot 5: Full hostel occupancy report.

```
Choose: 4

===== OCCUPANCY REPORT =====

Block A
A101: 1/4 occupied
A102: 0/4 occupied
A103: 0/2 occupied

Block B
B101: 0/4 occupied
B102: 0/4 occupied
B103: 0/2 occupied

Block C
C101: 0/3 occupied
C102: 0/3 occupied
C103: 0/2 occupied
```

Screenshot 6: Fee defaulters report.

```
Choose: 5

Enter balance threshold: 100000

===== FEE DEFAULTERS =====
John Okello | VU001 | Balance: 300000.00
```

Screenshot 7: Save and exit message.

```
===== OCCUPANCY OVERVIEW =====
Block A: 0/10 occupied
Block B: 0/10 occupied
Block C: 0/8 occupied

===== HOSTEL MANAGEMENT SYSTEM =====
1. Register student
2. Record payment
3. Search student
4. Occupancy report
5. Fee defaulters
6. Save and exit

Choose:
```

These screenshots demonstrate the major functions of the completed system.

17. Conclusion

The Hostel Room Booking and Fees Management System provides a simple computerized solution for managing hostel accommodation and student fees.

The system successfully handles room allocation, student records, fee payments, balance tracking, searching, occupancy reporting, defaulter identification and file storage. It reduces errors associated with paper records and makes hostel management easier and more organized.

GROUP REPORT

HOSTEL ROOM BOOKING AND FEES MANAGEMENT SYSTEM

1. Introduction

The Hostel Room Booking and Fees Management System is a Python-based system developed to manage student hostel rooms and fees. It provides a simple computerized alternative to paper records.

2. Objectives

The system was developed to register students, allocate rooms according to capacity, record payments, calculate balances, search student records, produce occupancy reports, identify fee defaulters and save records.

3. System Implementation

The program uses Python dictionaries and lists to store hostel and student information. Separate functions handle registration, payments, searching, reporting, saving and loading. Records are stored in `hostel_data.txt` using JSON.

4. Main Functions

The system registers students and allocates them to available rooms after checking room capacity. It records full or partial payments and calculates the remaining balance.

Users can search by name or registration number. The system also produces occupancy information and identifies students whose outstanding balances are above a selected threshold.

A menu brings the main operations together, including registration, payment, search, occupancy report, fee defaulters, and save and exit.

5. Sample Results

Four sample students were used in the practical. Each had a hostel fee of 600,000. The recorded balances were 250,000 for Peter Ouma, 150,000 for Grace Achieng, 100,000 for Brian Kato and 600,000 for Jane Nakato.

With a defaulter threshold of 200,000, Peter Ouma and Jane Nakato were identified. The occupancy report also showed the spaces available in the hostel rooms.

6. Group Members and Contribution

Member	Registration Number	Contribution
Namakula Waludah	VU-DIT-2603-0250-EVE	Hostel blocks, rooms and room allocation
Obentu Elivis	VU-BCS-2603-203-EVE	Student registration and student search
Nyango George	VU-BIT-2603-3365	Fee payment and balance calculation
Mukiza Eugene	VU-DIT-2603-2687-EVE	Reports, file persistence and system integration

7. Challenges and Solutions

Room capacity was controlled by checking occupancy before allocation. Partial payments were handled by storing payments and calculating the remaining balance.

To reduce loss of records, student and hostel information is saved to a file and loaded when the program starts. Error handling was included for damaged data files.

8. Conclusion

The completed system provides a simple way to manage hostel accommodation and fees. It combines room allocation, student records, payments, searching, reporting and data storage in one Python program.

9. GitHub

<https://github.com/euginemukiza/Hostel-Room-Booking-and-Fees-Management-System>

The screenshot shows a web browser with multiple tabs. The active tab is the GitHub repository page for 'Hostel-Room-Booking-and-Fees-Management-System' by user euginemukiza. The browser's address bar shows the repository URL. The GitHub interface includes a navigation bar with links for Code, Issues, Pull requests, Agents, Actions, Projects, Wiki, Security and quality, and Insights. The repository name is displayed prominently, along with a 'Public' label and a 'Pin' button. Below the repository name, there is a section for branches and tags, showing 'main' as the selected branch, 1 branch, and 0 tags. A search bar for files is also present. The commit history section shows two commits: an 'Initial commit' for 'README.md' made 10 minutes ago, and a commit 'Add files via upload' for 'hostel_management_fixed.py' made 6 minutes ago. The commit hash '8f8b4e8' is visible next to the second commit.

main 1 Branch 0 Tags

Go to file T Add file <> Code

euginemukiza Add files via upload 8f8b4e8 · 6 minutes ago 2 Commits

README.md	Initial commit	10 minutes ago
hostel_management_fixed.py	Add files via upload	6 minutes ago

README